

Overview

Synchronous, asynchronous, and reactive execution models; Project Reactor; Spring WebFlux; backpressure; executor tuning; JMH benchmarking; k6 load tests; GC tuning; async-profiler.

Execution Model Comparison

Model	Class	Thread Behaviour	Use When
Synchronous	OrderApplicationService.placeOrder()	Caller thread blocks until done	Business-critical, needs ACID
Async (Completable)	NotificationService.sendAsync()	Returns immediately; runs in parallel	Fire-and-forget, side effects
Semi-sync	InventoryReservationService	Waits up to 2s then cancels	Timeout-bounded external call
Reactive (Mono/Flux)	CartReactiveService, ProductCatalogReactiveService	Non-blocking event loop	MongoDB / Redis reactive reads
Virtual Threads (Java 21)	HTTP request handling	Carrier thread per blocking I/O	DB-heavy, high concurrency

Thread Pool Configuration (AsyncConfig)

Pool	corePoolSize	maxPoolSize	queueCapacity	Used By
notificationExecutor	5	20	100	@Async("notificationExecutor") NotificationService
reportExecutor	2	5	50	Heavy report generation tasks
VirtualThreadExecutor (Java 21)		Unlimited	—	HTTP request handling (spring.threads.virtual.enabled)

Backpressure in Spring WebFlux

```
Flux.fromIterable(products)
    .delayElements(Duration.ofMillis(10)) // throttle
    .onBackpressureBuffer(500, // bounded buffer
        product -> log.warn("Buffer full, dropping {}", product.id()))
```

JMH Benchmarks

Benchmark	Mode	Compares
ProductSearchBenchmark	Throughput	JPA LIKE vs PostgreSQL FTS vs Elasticsearch
OrderBuilderBenchmark	Throughput	new Order(...) vs Order.Builder pattern
BatchInsertBenchmark	Throughput	saveAll() vs JDBC batchUpdate() for 10k records

Load Test Targets (SLOs)

Test	Tool	Users	Duration	SLO
login→cart→checkout flow	k6	10→100 VUs ramp	5 min	p99 < 500ms, error < 0.1%
Product search	JMeter	200 concurrent	5 min	p99 < 200ms